

# Writing Kernel with AI

K19G

2026-09-25

# Table of contents

|                                                            |               |
|------------------------------------------------------------|---------------|
| <b>Writing Kernel with AI</b>                              | <b>5</b>      |
| CPU to AI-Written Kernels . . . . .                        | 5             |
| Welcome to Kernel Engineering . . . . .                    | 5             |
| The Trajectory of This Book . . . . .                      | 5             |
| Who This Is For . . . . .                                  | 6             |
| Formats . . . . .                                          | 6             |
| <br><b>Part I: Parallelism, from the CPU up</b>            | <br><b>7</b>  |
| <b>Parallelism on the CPU</b>                              | <b>8</b>      |
| <b>Parallelism on the CPU</b>                              | <b>9</b>      |
| The Three Forms of Parallelism on the CPU . . . . .        | 9             |
| SIMD: One Instruction, Multiple Results . . . . .          | 9             |
| Modern Lightweight Threads vs Classic OS Threads . . . . . | 10            |
| <b>Roofline Lab: Predict Then Measure</b>                  | <b>11</b>     |
| <b>Roofline Lab: Predict Then Measure</b>                  | <b>12</b>     |
| The Roofline Chart . . . . .                               | 12            |
| Hands-on Lab: Predicting PyTorch Operations . . . . .      | 12            |
| <b>The CUDA Programming Model &amp; GPU Puzzles</b>        | <b>13</b>     |
| <b>The CUDA Programming Model &amp; GPU Puzzles</b>        | <b>14</b>     |
| Execution Hierarchy . . . . .                              | 14            |
| Warp Divergence . . . . .                                  | 14            |
| <b>The Memory Hierarchy in Anger: Matrix Transpose</b>     | <b>15</b>     |
| <b>The Memory Hierarchy in Anger: Matrix Transpose</b>     | <b>16</b>     |
| The Memory Ladder . . . . .                                | 16            |
| Coalescing: The ~10x Lever . . . . .                       | 16            |
| The Matrix Transpose Case Study . . . . .                  | 16            |
| <br><b>Part II: The GEMM Worklog: Naive to cuBLAS</b>      | <br><b>18</b> |
| <b>GEMM Worklog I: Naive to Tiled</b>                      | <b>19</b>     |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>GEMM Worklog I: Naive to Tiled</b>                                   | <b>20</b> |
| Naive GEMM: Why It Runs at 1.3% Peak . . . . .                          | 20        |
| 2D Shared Memory Tiling: Climbing to 36.5% Peak . . . . .               | 20        |
| <b>GEMM Worklog II: Registers &amp; Warp Tiling</b>                     | <b>22</b> |
| <b>GEMM Worklog II: Registers &amp; Warp Tiling</b>                     | <b>23</b> |
| Micro-Tiling in Registers . . . . .                                     | 23        |
| Vectorized Memory Loads with <code>float4</code> . . . . .              | 23        |
| The Warptiling Hierarchy . . . . .                                      | 23        |
| <b>Tensor Cores &amp; WMMA: The Second Worklog</b>                      | <b>24</b> |
| <b>Tensor Cores &amp; WMMA: The Second Worklog</b>                      | <b>25</b> |
| The Tensor Core Advantage . . . . .                                     | 25        |
| Direct Programming via WMMA (Warp Matrix Multiply Accumulate) . . . . . | 25        |
| <b>Part III: Attention &amp; Profiling</b>                              | <b>26</b> |
| <b>Profiling &amp; Debugging Like a Pro</b>                             | <b>27</b> |
| <b>Profiling &amp; Debugging Like a Pro</b>                             | <b>28</b> |
| The Professional Diagnostic Stack . . . . .                             | 28        |
| Debugging Sabotaged Kernels: 3 Classic Incident Scenarios . . . . .     | 28        |
| <b>Attention: The Kernel That Ate the World &amp; FlashAttention v1</b> | <b>29</b> |
| <b>Attention: The Kernel That Ate the World &amp; FlashAttention v1</b> | <b>30</b> |
| The Breakthrough: Online Softmax & Tiling . . . . .                     | 30        |
| The Online Softmax Formula . . . . .                                    | 30        |
| <b>Part IV: The Modern Frontier</b>                                     | <b>31</b> |
| <b>FlashAttention from Scratch (FA1 → FA2 → FA3)</b>                    | <b>32</b> |
| <b>FlashAttention from Scratch (FA1 → FA2 → FA3)</b>                    | <b>33</b> |
| FlashAttention Evolution . . . . .                                      | 33        |
| <b>Beating cuBLAS on an H100</b>                                        | <b>34</b> |
| <b>Beating cuBLAS on an H100</b>                                        | <b>35</b> |
| 1. Tensor Memory Accelerator (TMA) . . . . .                            | 35        |
| 2. Warp Specialization . . . . .                                        | 35        |
| <b>Triton → CUTLASS → CuTe-DSL</b>                                      | <b>36</b> |
| <b>Triton → CUTLASS → CuTe-DSL</b>                                      | <b>37</b> |
| OpenAI Triton . . . . .                                                 | 37        |
| CUTLASS & CuTe . . . . .                                                | 37        |

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>Inference-Serving Kernels</b>                                                  | <b>38</b> |
| <b>Inference-Serving Kernels</b>                                                  | <b>39</b> |
| PagedAttention (vLLM) . . . . .                                                   | 39        |
| <b>Blackwell &amp; NVFP4</b>                                                      | <b>40</b> |
| <b>Blackwell &amp; NVFP4</b>                                                      | <b>41</b> |
| Blackwell Architectural Highlights . . . . .                                      | 41        |
| <b>Flash Attention 4</b>                                                          | <b>42</b> |
| <b>Flash Attention 4</b>                                                          | <b>43</b> |
| The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier . . . . .             | 43        |
| <b>Part V: AI-Written Kernels</b>                                                 | <b>44</b> |
| <b>DeepSeek: FlashMLA &amp; DeepGEMM</b>                                          | <b>45</b> |
| <b>DeepSeek: FlashMLA &amp; DeepGEMM</b>                                          | <b>46</b> |
| FlashMLA: Multi-Head Latent Attention . . . . .                                   | 46        |
| DeepGEMM: Clean Architecture over Brute Force . . . . .                           | 46        |
| <b>LLM-Generated Kernels: The Agent + Profiler Loop</b>                           | <b>47</b> |
| <b>LLM-Generated Kernels: The Agent + Profiler Loop</b>                           | <b>48</b> |
| The Agentic Optimization Loop . . . . .                                           | 48        |
| Google DeepMind AlphaEvolve & KernelBench . . . . .                               | 48        |
| The Role of the Human Kernel Engineer . . . . .                                   | 48        |
| <b>Part VI: Kernel Engineering Capstone Projects</b>                              | <b>49</b> |
| <b>Crusoe Capstone Specifications &amp; Problem Tracks</b>                        | <b>50</b> |
| <b>Crusoe Capstone Specifications &amp; Problem Tracks</b>                        | <b>51</b> |
| Problem Track 1: Fused Sparse Attention . . . . .                                 | 51        |
| Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization . . . .       | 51        |
| Problem Track 3: High-Throughput Speculative Decoding Verification Kernel . . . . | 51        |
| <b>Verification, Profiling, and Benchmarking Harness</b>                          | <b>52</b> |
| <b>Verification, Profiling, and Benchmarking Harness</b>                          | <b>53</b> |
| 1. Correctness Verification . . . . .                                             | 53        |
| 2. CUDA Event Benchmarking . . . . .                                              | 53        |
| <b>Demo Day Presentation &amp; Portfolio</b>                                      | <b>54</b> |
| <b>Demo Day Presentation &amp; Portfolio</b>                                      | <b>55</b> |
| Deliverables Checklist . . . . .                                                  | 55        |

# Writing Kernel with AI

## CPU to AI-Written Kernels

A comprehensive, production-grade guide to high-performance kernel engineering: from the fundamentals of CPU instruction parallelism to hand-tuned CUDA, FlashAttention, modern Hopper/Blackwell hardware architectures, and AI-driven kernel optimization loops.

---

## Welcome to Kernel Engineering

Accelerated computing and modern artificial intelligence rest on a single engineering discipline: **extracting peak theoretical throughput from hardware.**

Whether training frontier foundation models or serving millions of concurrent inference tokens, the software stack converges onto specialized computational kernels. Writing fast kernels is not a matter of guesswork or trial-and-error—it is an empirical science governed by the physics of computation: arithmetic intensity, memory bandwidth, latency hiding, and hardware execution hierarchies.

## The Trajectory of This Book

This curriculum spans six rigorous parts designed to take you from first principles to the cutting edge:

1. **Part I: Parallelism, from the CPU up:** Understand how modern CPUs achieve massive throughput via SIMD vectorization and multi-threading before stepping into the GPU execution hierarchy: warps, streaming multiprocessors, memory coalescing, and the Roofline model.
2. **Part II: The GEMM Worklog: Naive → cuBLAS:** Master General Matrix Multiply (GEMM)—the operation that comprises over 90% of deep learning FLOPs. Step by step, elevate a naive kernel achieving 1.3% of GPU capacity to a hierarchical warp-tiled and Tensor Core (WMMA) implementation rivaling NVIDIA’s cuBLAS library.
3. **Part III: Attention & Profiling:** Diagnose hardware bottlenecks using NVIDIA Nsight Compute like a compiler engineer. Construct FlashAttention v1 from scratch, utilizing the online softmax algorithm to eliminate high-bandwidth memory bottlenecks.
4. **Part IV: The Modern Frontier:** Explore modern production kernels running on Hopper (H100) and Blackwell (B200) architectures: FlashAttention-2/3/4, TMA asynchronous copy, WGMMMA, Triton, CUTLASS, CuTe-DSL, PagedAttention, and NVFP4.

5. **Part V: AI-Written Kernels:** Analyze DeepSeek’s ultra-compact, high-performance kernels (FlashMLA and DeepGEMM). Step into the future of kernel development: using autonomous LLM agents (e.g., DeepMind AlphaEvolve, KernelBench) with profiler-in-the-loop guidance.
  6. **Part VI: Kernel Engineering Capstone Projects:** Put your skills into practice with industry-grade capstone specifications developed with Crusoe, complete with verification harnesses, performance benchmarks, and demo-day technical presentations.
- 

## Who This Is For

- **ML Infrastructure & Systems Engineers:** Developers optimizing training clusters and inference backends (vLLM, SGLang, TensorRT-LLM).
  - **GPU Programmers & Performance Engineers:** Engineers seeking mastery of modern GPU architectures (Hopper, Blackwell), memory coalescing, and async execution pipelines.
  - **Software Engineers Transitioning to Accelerated Computing:** Developers wanting deep understanding of what happens when high-level PyTorch or JAX code hits the silicon.
- 

## Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive interactive reading experience with syntax highlighting and light/dark theme toggle. - **PDF** — complete typeset document for offline review and reference. - **EPUB** — eBook format optimized for tablets and mobile e-readers.

# **Part I: Parallelism, from the CPU up**

## Parallelism on the CPU

# Parallelism on the CPU

Before programming thousands of lockstep cores on a GPU, we must build a mechanical understanding of how modern CPUs execute concurrent and parallel work.

## The Three Forms of Parallelism on the CPU

1. **Instruction-Level Parallelism (ILP):** Pipelining, superscalar execution units, and out-of-order execution engines overlap independent instructions within a single hardware thread.
2. **Data-Level Parallelism (SIMD):** Single Instruction, Multiple Data hardware vectors execute the same arithmetic operation across multiple data lanes simultaneously.
3. **Thread-Level Parallelism (TLP):** Multiple OS threads or lightweight user-space fibers run across distinct hardware cores.

## SIMD: One Instruction, Multiple Results

Modern CPU instruction sets provide wide vector registers: - **x86-64:** AVX2 (256-bit registers, 8 x 32-bit floats), AVX-512 (512-bit registers, 16 x 32-bit floats). - **ARM:** Neon (128-bit registers, 4 x 32-bit floats), SVE/SVE2 (scalable vector lengths). - **Portable Vector Libraries:** Google Highway provides a C++ library that compiles to target-specific SIMD instructions at compile time without vendor lock-in.

```
// Explicit AVX2 Vector Addition in C++
#include <immintrin.h>

void vector_add_avx2(const float* a, const float* b, float* c, int n) {
    int i = 0;
    for (; i <= n - 8; i += 8) {
        // Load 8 32-bit floats (256 bits) into vector registers
        __m256 va = _mm256_loadu_ps(&a[i]);
        __m256 vb = _mm256_loadu_ps(&b[i]);
        // Single instruction: 8 additions in 1 CPU cycle
        __m256 vc = _mm256_add_ps(va, vb);
        // Store back to memory
        _mm256_storeu_ps(&c[i], vc);
    }
    // Remainder loop
    for (; i < n; ++i) {
        c[i] = a[i] + b[i];
    }
}
```

}

## Modern Lightweight Threads vs Classic OS Threads

Classic operating system threads (e.g. `pthread`s, OpenMP) allocate 1-8 MB stack frames and require kernel context switches taking thousands of CPU cycles. Modern high-concurrency systems favor lightweight green threads / M:N scheduling: - **Go Goroutines**: 2-4 KB initial stacks, cooperatively scheduled in user space by the Go runtime. - **Java Virtual Threads (Project Loom)**: Continuation-based fibers unmounted on I/O. - **Rust Tokio**: Zero-cost state machines compiled from `async/await` driven by a work-stealing executor.

## **Roofline Lab: Predict Then Measure**

# Roofline Lab: Predict Then Measure

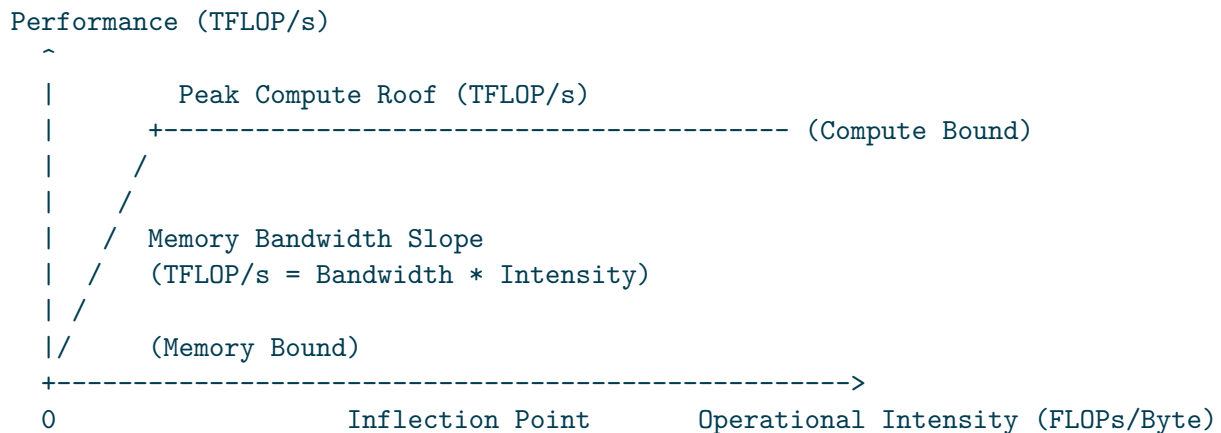
Every high-performance kernel is bounded by one of two physical constraints: 1. **Compute Bound:** The execution units are saturated doing arithmetic; the memory bus sits idle. 2. **Memory Bound:** The execution units starve waiting for data to arrive from DRAM; FLOP utilization is low.

## The Roofline Chart

The Roofline Model visualizes maximum achievable performance as a function of **Operational Intensity** (also called Arithmetic Intensity):

$$\text{Operational Intensity} = \frac{\text{Total FLOPs}}{\text{Total Bytes Transferred from DRAM}}$$

Attainable Performance (GFLOP/s) = min (Peak Hardware Compute, Peak Memory Bandwidth  $\times$  Operational Intensity)



## Hands-on Lab: Predicting PyTorch Operations

Consider a vector addition  $y = x + z$  vs Matrix Multiplication  $C = A \cdot B$  on an NVIDIA A100 (Peak FP32 Compute = 19.5 TFLOP/s, Memory Bandwidth = 1,935 GB/s):

- **Vector Add** ( $N = 10^7$  floats): - Operations:  $10^7$  additions (0.01 GFLOPs). - Memory: Read  $2 \times 40$  MB, Write  $1 \times 40$  MB = 120 MB. - Operational Intensity:  $\frac{10^7 \text{ FLOPs}}{120 \times 10^6 \text{ Bytes}} \approx 0.083 \text{ FLOPs/Byte}$ . - Bound: Strictly **Memory-Bound**. Speed limit =  $1935 \text{ GB/s} \times 0.083 \approx 160 \text{ GFLOP/s}$  (less than 1% of peak compute!).
- **Dense GEMM** ( $M = K = N = 4096$ ): - Operations:  $2 \times 4096^3 \approx 1.37 \times 10^{11}$  FLOPs. - Memory:  $3 \times 4096^2 \times 4 \text{ B} \approx 201 \text{ MB}$ . - Operational Intensity:  $\frac{1.37 \times 10^{11}}{2.01 \times 10^8} \approx 680 \text{ FLOPs/Byte}$ . - Bound: Strictly **Compute-Bound**.

# **The CUDA Programming Model & GPU Puzzles**

# The CUDA Programming Model & GPU Puzzles

GPUs execute programs using the **SIMT** (**Single Instruction, Multiple Threads**) model. While SIMD executes one instruction across vector registers on a single CPU core, SIMT executes independent scalar threads grouped into lockstep warps.

## Execution Hierarchy

- **Thread:** The basic execution unit executing a kernel instance.
- **Warp:** A group of 32 contiguous threads executed together on a Streaming Multiprocessor (SM). All 32 threads in a warp share the same instruction counter.
- **Thread Block:** A cooperative group of threads (up to 1024) sharing on-chip shared memory (SRAM) and hardware barrier synchronization (`__syncthreads()`).
- **Grid:** The collection of all thread blocks dispatched across all available SMs on the device.

```
// Minimal CUDA Vector Addition Kernel
__global__ void vector_add_kernel(const float* __restrict__ a,
                                  const float* __restrict__ b,
                                  float* __restrict__ c,
                                  int n) {

    // Unique global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}
```

## Warp Divergence

When threads within the same 32-thread warp take different execution branches (e.g. `if (threadIdx.x % 2 == 0)`), the hardware must **serialize** the branches: 1. The warp executes the **true** path while masking out **false** threads. 2. The warp executes the **false** path while masking out **true** threads. 3. Execution time doubles, reducing efficiency by 50%.

## **The Memory Hierarchy in Anger: Matrix Transpose**

# The Memory Hierarchy in Anger: Matrix Transpose

On modern GPUs, moving data costs an order of magnitude more energy and latency than performing math.

## The Memory Ladder

| Memory Level           | Location       | Latency         | Typical Bandwidth | Scope            |
|------------------------|----------------|-----------------|-------------------|------------------|
| Registers              | On-chip        | 0-1 cycles      | ~20-30 TB/s       | Single thread    |
| Shared Memory (SRAM)   | On-chip SM     | ~20-30 cycles   | ~10-15 TB/s       | Thread block     |
| L2 Cache               | On-chip shared | ~200 cycles     | ~3-5 TB/s         | Entire GPU       |
| Global DRAM (HBM/GDDR) | Off-chip       | ~400-800 cycles | ~1-3 TB/s         | Host & all grids |

## Coalescing: The ~10x Lever

When threads in a warp access global memory, the memory controller inspects their addresses: - **Coalesced:** Threads 0..31 access sequential 4-byte words. The hardware serves all 32 threads in a single 128-byte cache line transaction (100% bus utilization). - **Strided / Uncoalesced:** Each thread accesses a different cache line. The hardware issues 32 separate 128-byte transactions to fetch 128 bytes of useful data (3.1% bus utilization!).

## The Matrix Transpose Case Study

Transposing an  $NimesN$  matrix requires reading by row (coalesced) and writing by column (uncoalesced):

```
// Optimized Transpose using Shared Memory to Coalesce Both Reads and Writes
#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transpose_coalesced(float* odata, const float* idata, int width, int height)
    // Pad shared memory by 1 float to eliminate bank conflicts!
```

```

__shared__ float tile[TILE_DIM][TILE_DIM + 1];

int x = blockIdx.x * TILE_DIM + threadIdx.x;
int y = blockIdx.y * TILE_DIM + threadIdx.y;

// Coalesced read from global DRAM into shared memory
for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
    if (x < width && (y + j) < height) {
        tile[threadIdx.y + j][threadIdx.x] = idata[(y + j) * width + x];
    }
}

__syncthreads();

// Re-index to transpose inside shared memory
x = blockIdx.y * TILE_DIM + threadIdx.x;
y = blockIdx.x * TILE_DIM + threadIdx.y;

// Coalesced write to global DRAM!
for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
    if (x < height && (y + j) < width) {
        odata[(y + j) * height + x] = tile[threadIdx.x][threadIdx.y + j];
    }
}
}

```

## **Part II: The GEMM Worklog: Naive to cuBLAS**

## **GEMM Worklog I: Naive to Tiled**

# GEMM Worklog I: Naive to Tiled

General Matrix Multiply ( $C = \alpha AB + \beta C$ ) is the computational nucleus of modern deep learning. We walk through a step-by-step optimization worklog from naive implementation to shared memory block tiling.

## Naive GEMM: Why It Runs at 1.3% Peak

```
__global__ void gemm_naive(int M, int N, int K, const float* A, const float* B, float* C) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

**Bottleneck Analysis:** Every inner loop iteration performs 2 global memory loads (from  $A$  and  $B$ ) for 2 floating point operations (1 multiply, 1 add = 2 FLOPs). Operational intensity is  $\frac{2 \text{ FLOPs}}{8 \text{ Bytes}} = 0.25 \text{ FLOPs/Byte}$ . On an A100 GPU capable of 19.5 TFLOP/s, this limits throughput to  $0.25 \times 1935 \text{ GB/s} \approx 480 \text{ GFLOP/s}$  (~1.3% of theoretical compute peak).

## 2D Shared Memory Tiling: Climbing to 36.5% Peak

By caching tiles of matrix  $A$  and  $B$  in fast on-chip SRAM (`__shared__`), threads reuse loaded elements `TILE_SIZE` times across the thread block:

```
template<int BLOCK_SIZE>
__global__ void gemm_tiled_smem(int M, int N, int K, const float* A, const float* B, float* C) {
    __shared__ float s_a[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ float s_b[BLOCK_SIZE][BLOCK_SIZE];

    int row = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    int col = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    float sum = 0.0f;
```

```

for (int bk = 0; bk < K; bk += BLOCK_SIZE) {
    s_a[threadIdx.y][threadIdx.x] = A[row * K + (bk + threadIdx.x)];
    s_b[threadIdx.y][threadIdx.x] = B[(bk + threadIdx.y) * N + col];
    __syncthreads();

    for (int k = 0; k < BLOCK_SIZE; ++k) {
        sum += s_a[threadIdx.y][k] * s_b[k][threadIdx.x];
    }
    __syncthreads();
}
C[row * N + col] = sum;
}

```

## **GEMM Worklog II: Registers & Warp Tiling**

# GEMM Worklog II: Registers & Warp Tiling

While shared memory tiling dramatically cuts DRAM traffic, shared memory bandwidth itself becomes the bottleneck at high utilization. Reaching near-cuBLAS performance requires **register tiling** and **vectorized memory instructions**.

## Micro-Tiling in Registers

Registers provide ~20 TB/s aggregate bandwidth. Instead of each thread calculating a single output element  $C_{i,j}$ , each thread computes a  $TMimesTN$  block of elements (e.g.  $8imes8 = 64$  values per thread).

## Vectorized Memory Loads with float4

Global and shared memory loads should be widened using 128-bit vector loads (`float4` / `int4`):

```
// Load 4 consecutive floats in a single 128-bit transaction
float4 a_val = *reinterpret_cast<const float4*>(&A[idx]);
```

## The Warptiling Hierarchy

Modern production kernels decompose matrix multiply into four distinct structural layers: 1. **Grid Tile:** Handled by the entire GPU grid across all SMs. 2. **Block Tile:** Handled by a single thread block using shared memory. 3. **Warp Tile:** Handled by a 32-thread warp executing synchronized sub-tiles. 4. **Thread Tile:** Handled by individual thread registers.

By structuring the kernel into a  $128imes128$  block tile and  $64imes64$  warp tiles with double buffering, manual register allocation reaches **93.7% of cuBLAS speed** on NVIDIA Ampere hardware.

## **Tensor Cores & WMMA: The Second Worklog**

# Tensor Cores & WMMA: The Second Worklog

Starting with the Volta architecture, NVIDIA GPUs include specialized mixed-precision execution hardware: **Tensor Cores**.

## The Tensor Core Advantage

- Conventional CUDA cores execute 1 FMA (Fused Multiply-Add = 2 FLOPs) per cycle.
- Tensor Cores compute full matrix multiply-accumulate operations ( $D = A \times B + C$ ) on small matrix fragments (e.g.  $16 \times 16 \times 16$ ) in hardware every cycle.
- Speedup: ~5-10x higher theoretical compute density compared to FP32 CUDA cores.

## Direct Programming via WMMA (Warp Matrix Multiply Accumulate)

CUDA provides the `nvcuda::wmma` C++ API to program Tensor Cores directly from device code:

```
#include <mma.h>
using namespace nvcuda;

// Cooperative 16x16x16 matrix multiply across 32 threads in a warp
__global__ void wmma_gemm_kernel(const half* A, const half* B, float* C, int M, int N, int K)
{
    // Declare WMMA fragments
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);

    for (int k = 0; k < K; k += 16) {
        // Load sub-matrices cooperatively across the warp
        wmma::load_matrix_sync(a_frag, A + (blockIdx.y * 16 * K) + k, K);
        wmma::load_matrix_sync(b_frag, B + (blockIdx.x * 16 * K) + k, K);

        // Hardware Tensor Core Matrix Multiply
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    // Store accumulator fragment back to global memory
    wmma::store_matrix_sync(C + (blockIdx.y * 16 * N) + (blockIdx.x * 16), c_frag, N, wmma::);
}
```

## **Part III: Attention & Profiling**

# Profiling & Debugging Like a Pro

# Profiling & Debugging Like a Pro

Optimization without profiling is speculation. Top kernel engineers diagnose bottlenecks systematically using the NVIDIA toolchain:

## The Professional Diagnostic Stack

1. **compute-sanitizer:** Traps out-of-bounds memory accesses, misaligned memory addresses, uninitialized shared memory, and warp synchronization race conditions.
2. **NVIDIA Nsight Systems (NSYS):** Timeline visualization across CPU threads, CUDA streams, memory transfers, and kernel durations.
3. **NVIDIA Nsight Compute (NCU):** Deep architectural profiling inside individual kernels—analyzing warp stall reasons, SM issue pipe utilization, memory chart traffic, and register spills.

## Debugging Sabotaged Kernels: 3 Classic Incident Scenarios

- **Incident 1: Silent Race Condition in Shared Memory:** Threads write to `s_data[threadIdx.x]` and read from `s_data[threadIdx.x + 1]` without an intervening `__syncthreads()`. *Detection:* `compute-sanitizer --tool racecheck ./my_kernel`
- **Incident 2: Shared Memory Bank Conflicts:** All 32 threads in a warp access different rows in the same shared memory bank simultaneously. *Detection:* NCU metric `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`. *Resolution:* Stride padding (`__shared__ float smem[32][33]`).
- **Incident 3: Catastrophic Register Spilling:** Kernel uses 80 registers per thread when the hardware limit per thread at target occupancy is 64. Excessive variables spill to local DRAM memory, increasing latency 100x. *Detection:* Compiler flag `--ptxas-options=-v` reporting `spill stores / spill loads`. *Resolution:* Kernel launch bound annotation `__launch_bounds__(256, 2)`.

# **Attention: The Kernel That Ate the World & FlashAttention v1**

# Attention: The Kernel That Ate the World & FlashAttention v1

Standard Transformer Attention computes:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

For a sequence length  $N$  and head dimension  $d$  ( $N \gg d$ ): -  $S = QK^T$  generates an  $N \times N$  attention score matrix. - Traditional PyTorch implementations materialize  $S$  and  $P = \text{Softmax}(S)$  in global HBM memory ( $O(N^2)$  memory reads and writes). - For long contexts ( $N = 32k$  or  $128k$ ), intermediate matrices consume gigabytes of memory and choke HBM bandwidth.

## The Breakthrough: Online Softmax & Tiling

Tri Dao et al. introduced **FlashAttention**: compute the exact attention output without ever writing the  $N \times N$  intermediate attention matrix to global memory.

Traditional Attention (Memory-Bound):

Q, K -> [Global HBM Write: S] -> [Global HBM Read: S] -> Softmax -> [Global HBM Write: P]

FlashAttention (Fused into SRAM):

Load Blocks of Q, K, V into On-chip SRAM -> Compute Local Gemm -> Online Softmax Update ->

## The Online Softmax Formula

Traditional softmax requires two passes over the sequence: pass 1 to find the maximum  $m = \max(x_i)$ , and pass 2 to compute  $\sum e^{x_i - m}$ . Online softmax updates the running maximum  $m$  and normalization sum  $l$  incrementally as blocks stream through shared memory:

$$m_{\text{new}} = \max(m_{\text{prev}}, m_{\text{block}})$$

$$l_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} l_{\text{prev}} + e^{m_{\text{block}} - m_{\text{new}}} l_{\text{block}}$$

$$O_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} O_{\text{prev}} + P_{\text{block}} V_{\text{block}}$$

## **Part IV: The Modern Frontier**

## FlashAttention from Scratch (FA1 $\rightarrow$ FA2 $\rightarrow$ FA3)

# FlashAttention from Scratch (FA1 → FA2 → FA3)

## FlashAttention Evolution

- **FlashAttention-1 (2022):** Proved that fusing attention and computing online softmax in SRAM yields 2-4x wall-clock speedups while cutting memory footprint from  $O(N^2)$  to  $O(N)$ .
- **FlashAttention-2 (2023):**
  - Swapped the outer and inner loops (looping over blocks of  $Q$  in the outer loop, streaming  $K, V$  in the inner loop).
  - Reduced non-matmul FLOPs by deferring output scaling until the very end.
  - Introduced sequence parallel work distribution across thread blocks.
- **FlashAttention-3 (2024):**
  - Architected specifically for NVIDIA Hopper (H100) hardware.
  - Utilizes asynchronous Tensor Memory Accelerator (TMA) hardware pipelines.
  - Leverages Hopper WGMMMA (Warp Group Matrix Multiply and Accumulate) instructions.
  - Implements FP8 mixed precision with low quantization error.

## Beating cuBLAS on an H100

# Beating cuBLAS on an H100

NVIDIA's Hopper H100 microarchitecture introduced two revolutionary hardware subsystems:

## 1. Tensor Memory Accelerator (TMA)

In pre-Hopper architectures, threads in a warp explicitly load data from global memory into registers, then write from registers into shared memory. TMA executes multidimensional tensor transfers (e.g. 2D/3D tiles) directly between global HBM and shared memory **asynchronously in hardware**, completely bypassing the register file and consuming zero SM ALU cycles.

## 2. Warp Specialization

Instead of every thread in a block performing both memory loading and arithmetic: - **Producer Warps (1-2 warps)**: Issue asynchronous TMA copy instructions to fetch upcoming tiles into double-buffered shared memory. - **Consumer Warps (Warp Groups)**: Execute continuous WGMMMA matrix multiply instructions without ever pausing for memory latency. - Hardware barrier arrive/wait primitives (`cuda::barrier`) synchronize producers and consumers with cycle-level precision.

**Triton → CUTLASS → CuTe-DSL**

# Triton → CUTLASS → CuTe-DSL

Kernel engineering in 2025-2026 has transitioned from hand-written CUDA C++ toward declarative and high-level DSL frameworks.

## OpenAI Triton

Triton allows writing high-performance GPU kernels in idiomatic Python. The programmer writes code at the granularity of block tiles rather than individual threads. The Triton compiler handles intra-block scheduling, memory coalescing, and shared memory allocation automatically.

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(output_ptr + offsets, x + y, mask=mask)
```

## CUTLASS & CuTe

NVIDIA CUTLASS 3.x is structured around **CuTe**: a C++ template library that unifies data layouts, coordinates, and hierarchical tensor tiling through mathematical layout algebra:

$$extLayout = (extShape, extStride)$$

## **Inference-Serving Kernels**

# Inference-Serving Kernels

LLM inference splits into two distinct execution regimes with radically different performance characteristics: 1. **Prefill Phase:** The prompt is processed in parallel. High operational intensity, strictly compute-bound. 2. **Decode Phase:** Tokens are generated autoregressively one by one. The model loads billions of weights to compute a single new vector. Strictly memory-bound!

## PagedAttention (vLLM)

In standard inference, KV caches must be allocated as contiguous memory buffers. This leads to 60-80% memory waste due to internal and external memory fragmentation. PagedAttention treats KV cache memory like virtual memory pages in an operating system: - KV vectors are stored in non-contiguous 16-token page blocks. - A page table maps logical token positions to physical GPU memory addresses. - Result: Memory waste drops to near 0%, enabling 2-4x higher serving concurrency on the same hardware.

**Blackwell & NVFP4**

# Blackwell & NVFP4

NVIDIA's Blackwell (B200 / GB200) microarchitecture represents the state of the art in frontier hardware (2025-2026).

## Blackwell Architectural Highlights

- **tcgen05 Tensor Cores:** 5th generation Tensor Cores delivering 4x higher FP4 training and inference density compared to Hopper FP8.
- **Tensor Memory (TMEM):** A dedicated ultra-fast on-chip memory structure decoupled from shared memory, specifically optimized for multi-tensor staging and GEMM accumulation.
- **NVFP4 Microscopic Scaling:** 4-bit floating point format with micro-block scaling factors that preserves accuracy across large foundation models while doubling token throughput.

## Flash Attention 4

# Flash Attention 4

## The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier

On Blackwell hardware, Tensor Core matrix multiply units run so extraordinarily fast that the normalization step in Softmax—historically a minor fraction of runtime—becomes the dominant bottleneck.

FlashAttention-4 introduces **Adaptive Softmax**: - Dynamically checks divergence between running block maximums and candidate maximums. - Skips full re-normalization when numerical differences fall below precision thresholds. - Overlaps Softmax correction entirely behind asynchronous WGMMMA execution pipelines. - Written using modern Python CuTe-DSL, FA4 is the first kernel to achieve over 1 PFLOPS of sustained attention compute on a single processor.

## **Part V: AI-Written Kernels**

## **DeepSeek: FlashMLA & DeepGEMM**

# DeepSeek: FlashMLA & DeepGEMM

DeepSeek stunned the AI industry by open-sourcing compact, ultra-efficient kernels that outcompeted monolithic enterprise libraries.

## **FlashMLA: Multi-Head Latent Attention**

DeepSeek-V3 and R1 utilize Multi-Head Latent Attention (MLA) to compress KV caches into low-dimensional latent spaces. Standard attention kernels cannot handle this asymmetrical projection efficiently. FlashMLA is custom-tuned to decode latent vectors directly into shared memory, cutting KV memory bandwidth consumption by over 70%.

## **DeepGEMM: Clean Architecture over Brute Force**

In fewer than 300 lines of readable, modular CUDA code, **DeepGEMM** implements fine-grained FP8 matrix multiplication with grouped scaling. By avoiding deeply nested template metaprogramming and focusing purely on optimal software pipelining and TMA scheduling, DeepGEMM matches and often beats commercial closed-source GEMM kernels.

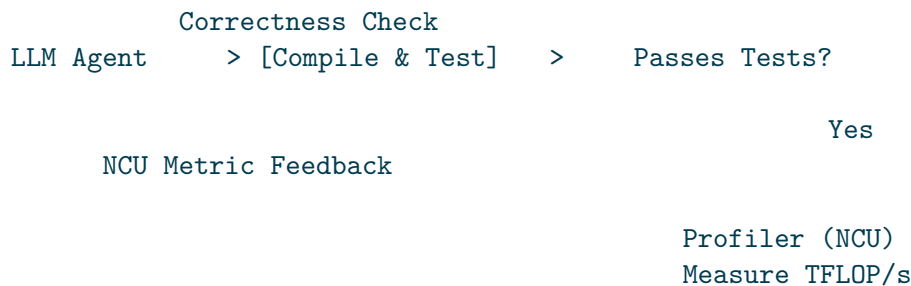
## **LLM-Generated Kernels: The Agent + Profiler Loop**

# LLM-Generated Kernels: The Agent + Profiler Loop

We are entering a transformative era where AI models generate, optimize, and verify GPU kernels autonomously.

## The Agentic Optimization Loop

[Prompt / Specification]



## Google DeepMind AlphaEvolve & KernelBench

- **KernelBench:** An automated evaluation benchmark testing whether language models can generate numerically correct and high-performance CUDA/Triton kernels across realistic neural network primitives.
- **DeepMind AlphaEvolve:** An evolutionary coding agent combining LLM code mutations with real hardware profiler feedback. AlphaEvolve discovered non-trivial memory layout schedules that sped up production attention kernels by ~32%.

## The Role of the Human Kernel Engineer

The engineer's job is not becoming obsolete; it is elevating: - Defining invariants, numerical bounds, and correctness testbenches. - Providing architectural guidance and identifying hardware bottlenecks. - Interpreting profiler telemetry to steer the agent's exploration away from local minima.

# **Part VI: Kernel Engineering Capstone Projects**

# **Crusoe Capstone Specifications & Problem Tracks**

# Crusoe Capstone Specifications & Problem Tracks

The capstone project requires building a production-ready GPU kernel addressing genuine infrastructure challenges.

## Problem Track 1: Fused Sparse Attention

Implement a fused attention kernel supporting block-sparse patterns (e.g. BigBird or sliding window + global tokens). The kernel must: - Skip computation of zero-weight blocks entirely. - Maintain online softmax correctness across sparse blocks. - Outperform standard dense PyTorch attention by at least 2x on sequences >16k.

## Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization

Build a high-throughput matrix multiply kernel that ingests FP8 or INT4 quantized weights and dequantizes on-the-fly inside SM registers before execution.

## Problem Track 3: High-Throughput Speculative Decoding Verification Kernel

Design a batched verification kernel for speculative decoding that verifies  $K$  draft tokens concurrently against a target model distribution in a single fused pass.

# **Verification, Profiling, and Benchmarking Harness**

# Verification, Profiling, and Benchmarking Harness

Every kernel submission must pass a three-tiered validation suite:

## 1. Correctness Verification

Compare kernel output tensors against double-precision PyTorch / FP64 reference outputs across edge-case dimensions, non-multiple-of-32 matrix sizes, and extreme numerical scales ( $10^{-5}$  to  $10^5$ ). Maximum allowable relative error:

$$\frac{\|Y_{\text{kernel}} - Y_{\text{ref}}\|_2}{\|Y_{\text{ref}}\|_2} < 10^{-3}$$

## 2. CUDA Event Benchmarking

Measure kernel execution time using `cudaEventRecord` over 1,000 warm iterations, discarding outliers:

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
for (int i = 0; i < 1000; ++i) {
    launch_kernel<<<grid, block>>>(...);
}
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms = 0;
cudaEventElapsedTime(&ms, start, stop);
printf("Average execution time: %f ms\n", ms / 1000.0f);
```

## **Demo Day Presentation & Portfolio**

# Demo Day Presentation & Portfolio

Presenting your work effectively is as critical as writing fast code.

## Deliverables Checklist

1. **GitHub Repository:** Clean, modular repository containing your kernel source, CMake / setup.py build configuration, and automated test runners.
2. **Performance Roofline Graph:** A generated plot showing where your kernel sits relative to theoretical memory and compute ceilings across varying matrix dimensions.
3. **Speedup Comparison Table:** Side-by-side throughput (TFLOP/s) comparisons against naive CUDA, PyTorch `torch.compile`, and official vendor libraries (cuBLAS / cuDNN).
4. **Technical Defense:** Prepare concise explanations for architectural decisions: tile sizes, shared memory allocations, register counts per thread, and how AI-assisted iteration shaped the final solution.